Laporan Proyek Berpikir Komputasional (WI1102)

Kelas K05
Dosen: Prama Setia Putra, S.Si., M.Si., Ph.D.



Disusun Oleh:

Jovan Nathaniel Permana (16026295)
Qamila Annur Firdausi (16026380)
Abira Mutia Salsabila (16026060)
Ezra Hadyan Wicaksono (16026055)

Fakultas FMIPA Institut Teknologi Bandung
Semester 1 Tahun 2026/2027

DAFTAR ISI

1 Dasar-dasar Berpikir Komputasional yang Digunakan **2**
- 1.1 Algoritma Newton-Raphson . . . 2
  - 1.1.1 Dekomposisi . . . 2
  - 1.1.2 Pengenalan Pola . . . 3
  - 1.1.3 Abstraksi . . . 3
- 1.2 Algoritma Pinjam Buku . . . 4
  - 1.2.1 Dekomposisi . . . 4
  - 1.2.2 Pengenalan Pola . . . 5
  - 1.2.3 Abstraksi . . . 6
- 1.3 Algoritma Labirin . . . 6
  - 1.3.1 Dekomposisi . . . 6
  - 1.3.2 Pengenalan Pola . . . 7
  - 1.3.3 Abstraksi . . . 7

2 Lampiran **8**
- 2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram . 8
  - 2.1.1 Algoritma Newton-Raphson . . . 8
  - 2.1.2 Algoritma Pinjam Buku . . . 10
  - 2.1.3 Algoritma Labirin . . . 11
- 2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram 16
  - 2.2.1 Algoritma Newton-Raphson . . . 16
  - 2.2.2 Algoritma Pinjam Buku . . . 18
  - 2.2.3 Algoritma Labirin . . . 19

3 Kode Program Simulasi **24**
- 3.1 Algoritma Newton-Raphson . . . 24
- 3.2 Algoritma Pinjam Buku . . . 27
- 3.3 Algoritma Labirin . . . 30

4 Berbagai Link **35**
- 4.1 Link Repositori GitHub . . . 35

1 Dasar-dasar Berpikir Komputasional yang Digunakan

1.1 Algoritma Newton-Raphson

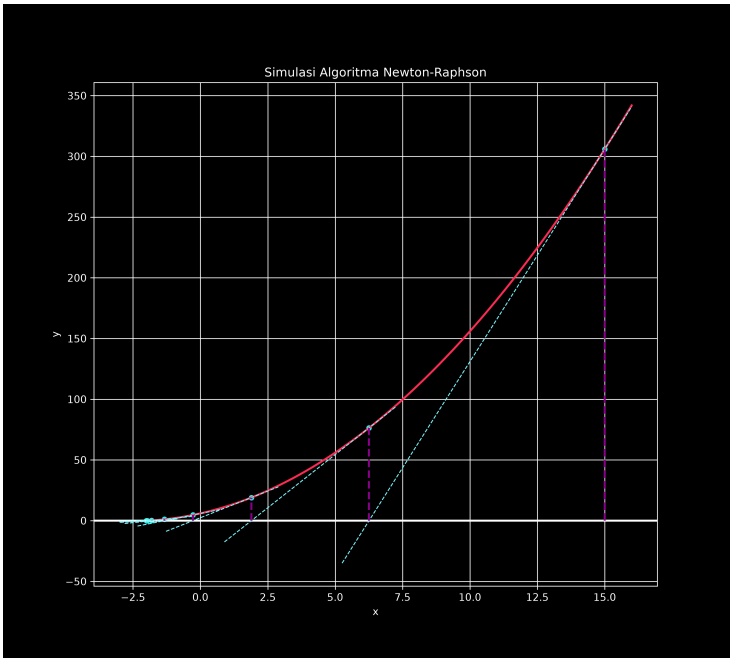


Figure 1: Gambar simulasi Newton-Raphson (untuk fungsi $f(x) = x^2+5x+6$ dengan tebakan awal $x_0 = 15$) menggunakan matplotlib

1.1.1 Dekomposisi

Algoritma Newton-Raphson sebenarnya adalah algoritma optimasi yang bersifat iteratif, dengan tujuan utama untuk mencari akar-akar/titik potong sebuah fungsi $f(x)$ dengan sumbu x. Dalam proyek ini, algoritma Newton-Raphson difokuskan untuk mencari akar-akar dari sebuah polinom dengan derajat tertentu, sesuai masukan dari pengguna (*user*). Melalui prinsip dekomposisi (pemecahan suatu masalah dengan skala besar menjadi skala yang lebih kecil), algoritma Newton-Raphson untuk mencari akar-akar sebuah fungsi polinom dapat dipecah menjadi beberapa tugas berikut.

1. Menginput derajat polinom (N), nilai koefisien-koefisiennya, serta nilai tebakan awal (x_0) dari user.

2. Untuk nilai tebakan awal $x = x_0$ yang diberikan, hitung nilai fungsi $f(x_0)$ beserta turunannya, yaitu $f'(x_0)$.

3. Nilai x diperbarui secara terus-menerus melalui prosedur iteratif dengan persamaan

$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)} \tag{1}$$

hingga nilai *error* yang terjadi dalam penentuan akar menjadi tidak signifikan atau berada di bawah ambang batas tertentu.

4. Jika $f'(x) = 0$ pada suatu tahap tertentu atau perulangan pada langkah 3 tidak dapat selesai (*infinite loop*), maka proses perhitungan akar gagal dilakukan.

1.1.2 Pengenalan Pola

Algoritma Newton-Raphson memiliki pola yang mirip dengan algoritma optimasi, yaitu algoritma yang menekankan perhitungan matematika secara iteratif (berulang) untuk mencapai suatu tujuan. Dalam algoritma Newton-Raphson, persamaan (1) dijalankan secara berulang-ulang untuk mencapai suatu tujuan, yaitu akar persamaan polinom. Selama perulangan tersebut, nilai x akan terus diperbarui (*update*) hingga error dalam pencarian akar menjadi tidak signifikan atau lebih kecil dari ambang batas tertentu.

1.1.3 Abstraksi

Algoritma Newton-Raphson membutuhkan beberapa informasi penting, yaitu:

1. Derajat polinom N.

2. Koefisien-koefisien pada polinom.

3. Nilai tebakan awal x_0.

Dalam proyek ini, derajat polinom N disimpan di dalam variabel bertipe `int`, sementara nilai tebakan awal x_0 disimpan di dalam variabel bertipe `float`. Nilai-nilai koefisien pada polinom disimpan di dalam sebuah array `float` yang berukuran $N + 1$.

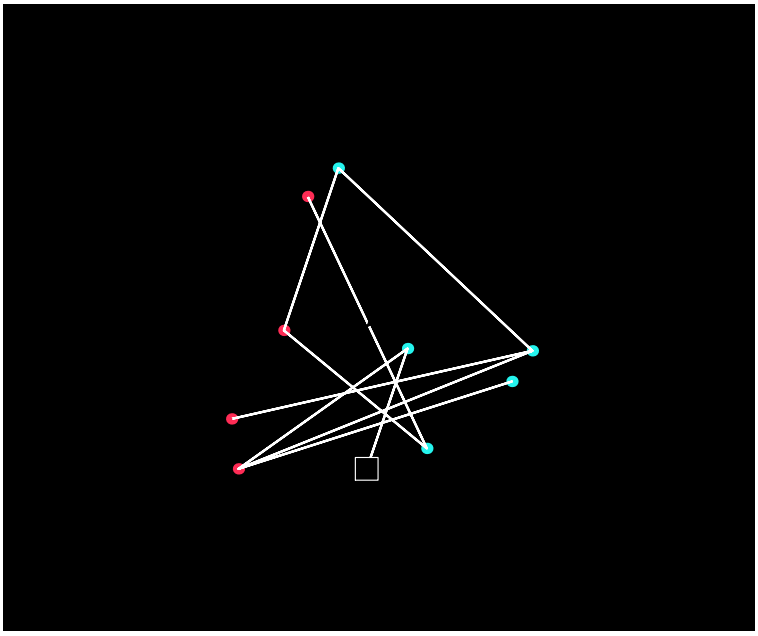


Figure 2: Gambar simulasi Pinjam Buku (untuk 10 orang peminjam) menggunakan Python Turtle

1.2 Algoritma Pinjam Buku

1.2.1 Dekomposisi

Algoritma pinjam buku merupakan algoritma berbasis teori graf (*graph theory*). Hubungan antar peminjam buku dapat direpresentasikan sebagai graf, dimana *nodes*/titik dari graf tersebut adalah peminjam dan *edges*/garis hubungan dari graf tersebut adalah hubungan antar peminjam. Graf yang dihasilkan adalah graf siklik (*cyclic graph*, dimana seseorang dapat bergerak dari satu *node* ke *node* lain, lalu kembali memutar ke titik semula) dengan *edge* dua arah. Selain itu, faktor umur yang diperhitungkan dalam algoritma pinjam buku (untuk menentukan peminjam berikutnya berdasarkan umur terendah) membuat graf ini menjadi graf berbobot (*weighted graph*). Oleh karena itu, dalam algoritma ini, digunakan algoritma DFS (*Depth First Search*) yang disertai dengan *backtracking*, dimana algoritma pinjam buku dapat dibagi menjadi beberapa tugas besar berikut.

1. Menginput matriks ketetanggaan (yang menggambarkan hubungan antar peminjam), umur setiap peminjam, dan peminjam pertama yang mendapatkan buku.

2. Selanjutnya, melalui matriks ketetanggaan, hubungan antara satu peminjam dengan peminjam lain dapat ditentukan. Apabila terdapat lebih dari satu peminjam, pilih peminjam dengan umur terendah dan prioritaskan peminjam yang belum pernah meminjam buku.

3. Jika buku sudah sampai di *node* buntu atau semua peminjam yang berhubungan dengan *node* tersebut sudah pernah meminjam buku sebelumnya, lakukan *backtracking*, yaitu mengembalikan buku ke peminjam sebelumnya.

4. Langkah 2 sampai 3 dilakukan terus-menerus sampai semua peminjam sudah pernah meminjam buku dan buku dikembalikan ke peminjam awal.

1.2.2 Pengenalan Pola

Pola yang umum terjadi pada algoritma DFS dan *backtracking* yang digunakan dalam algoritma pinjam buku adalah untuk setiap *node* yang dikunjungi, algoritma selalu mencari *node* berikutnya dengan umur terendah dan tidak pernah ada dalam riwayat kunjungan sebelumnya. Pernyataan tersebut menggambarkan bahwa algoritma selalu membuat keputusan terbaik secara lokal (dibuat berdasarkan *node* saat ini) untuk mencapai solusi global (semua orang sudah pernah meminjam buku sesuai dengan graf tiap-tiap orang). Hal ini merupakan salah satu sifat dari *greedy search algorithm*, yaitu algoritma yang selalu mengoptimalkan keputusan terbaik di setiap langkah untuk mencapai suatu solusi global. Namun, secara umum, *greedy search algorithm* tidak mempedulikan apakah suatu *node* sudah dikunjungi atau belum. Akan tetapi, algoritma pinjam buku harus menjamin bahwa semua orang sudah pernah meminjam buku dan buku dikembalikan ke peminjam awal, sehingga *backtracking* menjadi penting dalam kasus ini. Untuk melakukan *backtracking*, digunakan suatu array yang menyimpan histori *node* yang sudah dikunjungi. Array tersebut mula-mula kosong, tetapi seiring berjalannya algoritma dalam menelusuri setiap peminjam, array histori *node* tersebut akan ditumpuk dengan *node-node* yang sudah dikunjungi (*stacking*). Saat *backtracking*, tumpukan tersebut akan dikeluarkan mulai dari posisi teratas untuk menelusuri kembali peminjam buku sebelumnya dan menjamin bahwa semua peminjam sudah pernah meminjam buku dan buku kembali ke tempat asal.

1.2.3 Abstraksi

Informasi penting yang diperlukan dalam algoritma pinjam buku adalah sebagai berikut.

1. Matriks ketetanggaan (berukuran $N \times N$, dengan N adalah jumlah total peminjam). Matriks ini direpresentasikan sebagai array 2D bertipe `int`.
2. Peminjam pertama yang mendapatkan buku, direpresentasikan sebagai index bertipe `int`.
3. Umur setiap peminjam, disimpan dalam array bertipe `int` berukuran N.

1.3 Algoritma Labirin

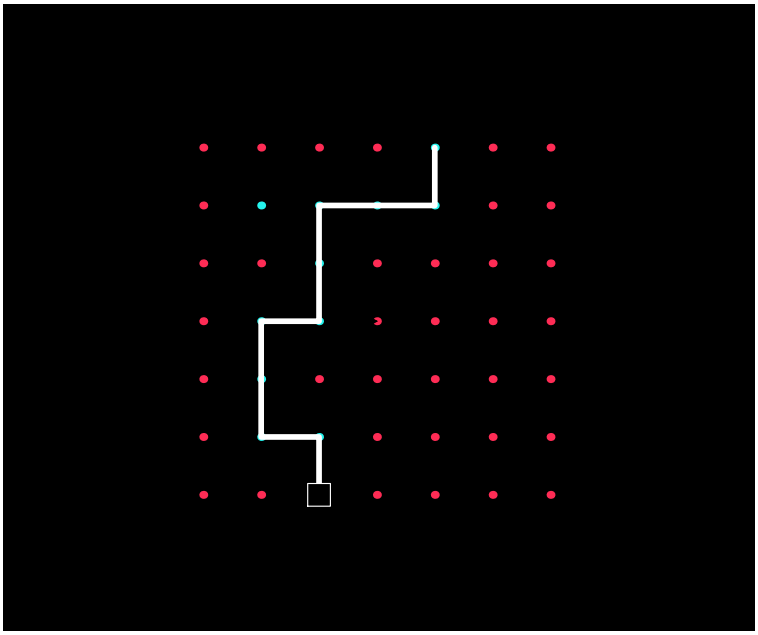


Figure 3: Gambar Simulasi Labirin Menggunakan Python Turtle

1.3.1 Dekomposisi

Algoritma labirin merupakan algoritma berbasis matriks dan pencarian berdasarkan arah. Dalam proyek ini, program di *setting* untuk menentukan titik awal perjalanan labirin terlebih dahulu kemudian *pointer* bergerak

(lewat) di jalur yang ditentukan dengan syarat khusus berdasarkan arah Utara, Timur, Selatan dan Barat yang kemudian arah putarnya akan berbalik saat jumlah jalannya habis dibagi keliling. Berbasis koordinat, output dari program ini adalah koordinat yang dilewati untuk berhasil mencapai ujung labirin. Melalui dekomposisi arah ini, algoritma labirin dapat dibagi menjadi beberapa tugas besar berikut.

1. Menginput matriks labirin (yang menggambarkan jalan (1) dan dinding labirin (0)), titik awal, batas gerak, *pointer*, dan besar matriks tersebut.

2. Selanjutnya, program akan mencari titik awal yang akan menjadi *pointer* dan akan bergerak ketika ada jalan (angka 1) berdasarkan arah Utara, Timur, Selatan dan Barat.

3. Program didesain dengan aturan *forbidden move* agar *pointer* tidak bolak balik di koordinat yang sudah pernah dilewati dan terus maju mencari jalan yang lain.

4. *Pointer* berubah arah menjadi Barat, Selatan, Timur, Utara ketika jalan buntu atau ada dinding (angka 0) dengan hitungan jumlah langkah habis dibagi keliling labirin.

5. Program akan mencatat koordinat langkah yang bisa dilewati lalu menampilkan alur jalan kepada user mulai dari titik awal hingga ujung labirin.

1.3.2 Pengenalan Pola

Algoritma labirin memiliki pola yang mirip seperti *searching algorithm* (spesifiknya *Breadth-First Search*), yaitu algoritma yang menjelajahi titik-titik tetangga terlebih dahulu sebelum lanjut ke langkah selanjutnya. Dalam algoritma labirin, *pointer* melangkah dalam matriks dengan mencari arah mata angin secara jarum jam, dan untuk mencegah *endless loops* maka setiap langkah ke-(sesuai keliling matriks), maka prioritas pencarian dibalik menjadi berlawanan arah jarum jam.

1.3.3 Abstraksi

Algoritma Labirin membutuhkan beberapa informasi penting, yaitu:

1. Ukuran labirin N.

2. Representasi labirin dalam bentuk matriks.

3. Penentuan awal dan titik akhir dari matriks labirin (*endpoints*)

Dalam proyek ini, ukuran labirin N disimpan di dalam variabel bertipe `int`, sementara representasi labirin dalam bentuk matriks disimpan dalam list dua dimensi yang berukuran $N \times N$. Posisi dari penunjuk koordinatnya (*pointer*) saat menelusuri labirin, arah gerakannya, status sisi, dan riwayat gerakannya disimpan ke dalam struktur list satu dimensi, kemudian program menjalankan proses perulangan secara otomatis untuk mencari jalur dari titik awal hingga mencapai titik akhir yang telah ditentukan sebelumnya dalam labirin.

2 Lampiran

2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram

2.1.1 Algoritma Newton-Raphson

Kode 1: Kode Python algoritma Newton-Raphson tanpa subprogram

```
import time
""" Newton Raphson (tanpa subprogram) """

# program utama
# input
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{
           i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
```

```
while True:
    k += 1
    val = 0.0
    el = 0
    element = 1
    while el <= (N-1):
        val += coefs[el] * (x ** el)
        el += 1
    grad = 0.0
    while element <= (N-1):
        grad += element * coefs[element] * (x ** (
            element - 1))
        element += 1

    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f}
        Gradien: {grad:.6f}')
    delta = x_new - x
    if delta < 0:
        delta = -delta

    if val > 0:
        absval = val
    else:
        absval = -val

    if (delta < epsilon) and (absval < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k
            } iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
```

```
63 print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.1.2 Algoritma Pinjam Buku

Kode 2: Kode Python algoritma Pinjam Buku tanpa subprogram

```
1  import time
2  """Pinjam Buku (Tanpa Subprogram)"""
3  # program utama
4  # input
5  N = int(input("Masukkan jumlah orang: "))
6  matrix = []
7  umur = [0 for _ in range(N)]
8  sudah_pinjam = [0 for _ in range(N)]
9  riwayat_pinjam = []
10 for i in range(N):
11     col = [0]*N
12     matrix += [col]
13
14 for y in range(N):
15     for x in range(N):
16         val = int(input(f'Masukkan isi matrix
               ketetanggaan untuk koordinat ({x},{y}): '))
17         matrix[y][x] = val
18     print('================================')
19
20 for i in range(N):
21     val_umur = int(input(f'Masukkan umur untuk index {i
          }: '))
22     umur[i] = val_umur
23
24 index_pinjam = int(input('Masukkan index tempat awal
       buku berada: '))
25 # proses
26 awal = time.perf_counter()
27 sudah_pinjam[index_pinjam] = 1
28 riwayat_pinjam += [index_pinjam]
29 top = 0
30 print('Urutan index peminjaman buku: ', end='')
31 print(index_pinjam, end='')
32 while True:
33     koneksi = matrix[index_pinjam]
34     # pindah peminjam
```

```
    i = 0
    kandidat = -1
    umur_min = 999999
    while i < N:
        # cek koneksi dan status pinjam
        if (koneksi[i] == 1) and (sudah_pinjam[i] == 0):
            # Cek umur termuda
            if umur[i] < umur_min:
                umur_min = umur[i]
                kandidat = i
        i += 1

    if kandidat != -1:
        index_pinjam = kandidat
        sudah_pinjam[index_pinjam] = 1
        riwayat_pinjam += [index_pinjam]
        top += 1
        print(" ->", index_pinjam, end="")
    else:
        # Pengecekan apakah semua orang sudah pernah
          meminjam
        jumlah = 0
        for items in sudah_pinjam:
            jumlah += items
        # selesai hanya jika semua orang sudah pernah
          meminjam dan buku kembali ke peminjam asal
        if top == 0 and jumlah == N:
            break
        # Jika belum selesai tapi mentok, backtrack ke
          orang sebelumnya
        top -= 1
        index_pinjam = riwayat_pinjam[top]
        print(" ->", index_pinjam, end="")

akhir = time.perf_counter()
eks = akhir - awal
print(f'\n\nWaktu eksekusi: {eks} detik.')
```

2.1.3 Algoritma Labirin

Kode 3: Kode Python algoritma Labirin tanpa subprogram

```
import time
```

```
"""labirin (tanpa Subprogram)"""
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Apakah koordinat ({x},{y})
            bisa dilewati (1) atau tidak (0): '))
        matrix[y][x] = val
    print('================================')

# proses
start_time = time.perf_counter()
# inisialisasi pointer dan arahnya
status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
# cari titik awal di sisi atas
for i in range(N):
    if matrix[0][i] == 1:
        status_sisi[0] = [i,0]
        break
# cari titik awal di sisi bawah
for i in range(N):
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
```

```
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
        direction = [1,1,1,1]

    # cek apakah bisa dilewati
    i = 0
    while(i < 4):
        if direction[i] == 1:
            if (i == 0) and (forbidden_move[i] == 0):
```

```
83                  bukan_dinding = int(matrix[pointer
                        [1]-1][pointer[0]] == 1)
84                  avail_move[i] = bukan_dinding
85              elif (i == 1) and (forbidden_move[i] == 0):
86                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]+1] == 1)
87                  avail_move[i] = bukan_dinding
88              elif (i == 2) and (forbidden_move[i] == 0):
89                  bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
90                  avail_move[i] = bukan_dinding
91              elif (i == 3) and (forbidden_move[i] == 0):
92                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]-1] == 1)
93                  avail_move[i] = bukan_dinding
94          else:
95              avail_move[i] = 0
96          i += 1
97      # Jika jalan buntu
98      if avail_move == [0,0,0,0]:
99          i = 0
100         while(i < 4):
101             if direction[i] == 1:
102                 if i == 0:
103                     bukan_dinding = int(matrix[pointer
                            [1]-1][pointer[0]] == 1)
104                     avail_move[i] = bukan_dinding
105                 elif i == 1:
106                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]+1] == 1)
107                     avail_move[i] = bukan_dinding
108                 elif i == 2:
109                     bukan_dinding = int(matrix[pointer
                            [1]+1][pointer[0]] == 1)
110                     avail_move[i] = bukan_dinding
111                 else:
112                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]-1] == 1)
113                     avail_move[i] = bukan_dinding
114             else:
115                 avail_move[i] = 0
116             i += 1
117
```

```
    # Pilih jalur
    if clockwise:
        j = 0
        while (j < 4):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j += 1
    else:
        j = 3
        while (j >= 0):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j -= 1
```

```
161
162     if (change_clockwise % (4*N)) == 0:
163         clockwise = not clockwise
164
165 print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
    dilewati')
166 end_time = time.perf_counter()
167 execution_time = end_time - start_time
168 print('Labirin Diselesaikan!')
169 print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram

2.2.1 Algoritma Newton-Raphson

Kode 4: Kode Python algoritma Newton-Raphson dengan subprogram

```
1  import time
2  """ Newton Raphson (dengan subprogram) """
3
4  # definisi fungsi
5  def function(x, coefs):
6      value = 0.0
7      for i in range(len(coefs)):
8          value += coefs[i] * (x**i)
9      return value
10
11 def gradient(x, coefs):
12     value = 0.0
13     for i in range(1, len(coefs)):
14         value += i * coefs[i] * (x**(i - 1))
15     return value
16
17 def absol(x):
18     value = (x >= 0) * x + (x < 0) * (-x)
19     return value
20
21 # program utama
22 # input
23 N = int(input('Masukkan derajat polinomial: '))
24 N += 1
25 coefs = [0]*N
```

```
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')

    if (absol(x_new - x) < epsilon) and (absol(val) < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2.2 Algoritma Pinjam Buku

Kode 5: Kode Python algoritma Pinjam Buku dengan subprogram

```python
import time
"""Pinjam Buku (dengan Subprogram)"""
# definisi fungsi
def cari_kandidat(koneksi, umur, sudah_pinjam, N):
    # pindah peminjam
    i = 0
    kandidat = -1
    umur_min = 999999
    while i < N:
        # cek koneksi dan status pinjam
        if (koneksi[i] == 1) and (sudah_pinjam[i] == 0):
            # Cek umur termuda
            if umur[i] < umur_min:
                umur_min = umur[i]
                kandidat = i
        i += 1
    return kandidat
# program utama
# input
N = int(input("Masukkan jumlah orang: "))
matrix = []
umur = [0 for _ in range(N)]
sudah_pinjam = [0 for _ in range(N)]
riwayat_pinjam = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Masukkan isi matrix
           ketetanggaan untuk koordinat ({x},{y}): '))
        matrix[y][x] = val
    print('================================')

for i in range(N):
    val_umur = int(input(f'Masukkan umur untuk index {i
       }: '))
    umur[i] = val_umur

```

```
index_pinjam = int(input('Masukkan index tempat awal buku berada: '))
# proses
awal = time.perf_counter()
sudah_pinjam[index_pinjam] = 1
riwayat_pinjam += [index_pinjam]
top = 0
print('Urutan index peminjaman buku: ', end='')
print(index_pinjam, end='')
while True:
    koneksi = matrix[index_pinjam]
    kandidat = cari_kandidat(koneksi, umur, sudah_pinjam, N)
    if kandidat != -1:
        index_pinjam = kandidat
        sudah_pinjam[index_pinjam] = 1
        riwayat_pinjam += [index_pinjam]
        top += 1
        print(" ->", index_pinjam, end="")
    else:
        # Pengecekan apakah semua orang sudah pernah meminjam
        jumlah = 0
        for items in sudah_pinjam:
            jumlah += items
        # selesai hanya jika semua orang sudah pernah meminjam dan buku kembali ke peminjam asal
        if top == 0 and jumlah == N:
            break
        # Jika belum selesai tapi mentok, backtrack ke orang sebelumnya
        top -= 1
        index_pinjam = riwayat_pinjam[top]
        print(" ->", index_pinjam, end="")

akhir = time.perf_counter()
eks = akhir - awal
print(f'\n\nWaktu eksekusi: {eks} detik.')
```

2.2.3 Algoritma Labirin

Kode 6: Kode Python algoritma Labirin dengan subprogram

```python
import time
"""labirin (dengan Subprogram)"""
# definisi fungsi
def choose_dir(pointer):
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
        direction = [1,1,1,1]
    return direction
# program utama
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Apakah koordinat ({x},{y})
           bisa dilewati (1) atau tidak (0): '))
        matrix[y][x] = val
    print('================================')

# proses
start_time = time.perf_counter()
```

```
# inisialisasi pointer dan arahnya
status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
# cari titik awal di sisi atas
for i in range(N):
    if matrix[0][i] == 1:
        status_sisi[0] = [i,0]
        break
# cari titik awal di sisi bawah
for i in range(N):
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    direction = choose_dir(pointer)
```

```
    # cek apakah bisa dilewati
    i = 0
    while(i < 4):
        if direction[i] == 1:
            if (i == 0) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                    [1]-1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 1) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                    pointer[0]+1] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 2) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                    [1]+1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 3) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                    pointer[0]-1] == 1)
                avail_move[i] = bukan_dinding
        else:
            avail_move[i] = 0
        i += 1
    # Jika jalan buntu
    if avail_move == [0,0,0,0]:
        i = 0
        while(i < 4):
            if direction[i] == 1:
                if i == 0:
                    bukan_dinding = int(matrix[pointer
                        [1]-1][pointer[0]] == 1)
                    avail_move[i] = bukan_dinding
                elif i == 1:
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]+1] == 1)
                    avail_move[i] = bukan_dinding
                elif i == 2:
                    bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
                    avail_move[i] = bukan_dinding
                else:
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]-1] == 1)
```

```
                    avail_move[i] = bukan_dinding
            else:
                avail_move[i] = 0
            i += 1

    # Pilih jalur
    if clockwise:
        j = 0
        while (j < 4):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j += 1
    else:
        j = 3
        while (j >= 0):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
```

```
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j -= 1

    if (change_clockwise % (4*N)) == 0:
        clockwise = not clockwise

print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
   dilewati')
end_time = time.perf_counter()
execution_time = end_time - start_time
print('Labirin Diselesaikan!')
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

3 Kode Program Simulasi

3.1 Algoritma Newton-Raphson

Kode 7: Kode Python simulasi algoritma Newton-Raphson

```
import time
import matplotlib.pyplot as plt
import numpy as np
""" Newton Raphson (Simulasi) """
plt.style.use('dark_background')
# definisi fungsi
def function(x, coefs):
    value = 0.0
    for i in range(len(coefs)):
        value += coefs[i] * (x**i)
    return value

def gradient(x, coefs):
    value = 0.0
    for i in range(1, len(coefs)):
        value += i * coefs[i] * (x**(i - 1))
    return value

def absol(x):
    value = (x >= 0) * x + (x < 0) * (-x)
```

```
21      return value
22
23  def gradient_line_gen(x, x1, coefs):
24      return gradient(x1, coefs)*x - gradient(x1, coefs)*
            x1 + function(x1, coefs)
25
26  def cari_batas_kiri(grad_lines):
27      minimums = []
28      for i in range(len(grad_lines)):
29          minimums.append(min(grad_lines[i][0]))
30      return min(minimums)
31
32  def cari_batas_kanan(grad_lines):
33      maximums = []
34      for i in range(len(grad_lines)):
35          maximums.append(max(grad_lines[i][0]))
36      return max(maximums)
37
38  # program utama
39  # input
40  N = int(input('Masukkan derajat polinomial: '))
41  N += 1
42  coefs = [0]*N
43  for i in range(N):
44      if(i == 0):
45          val = float(input('Masukkan nilai konstanta: '))
46      else:
47          val = float(input(f'Masukkan nilai koefisien x^{
                i}: '))
48
49      coefs[i] = val
50
51  x_init = float(input('Masukkan tebakan awal x: '))
52  x = x_init
53  gradient_lines = []
54  absis_singgung = []
55  ordinat_singgung = []
56  epsilon = 1e-4
57  k = 0
58  max_iter = 1e6
59  # proses iterasi
60  start_time = time.perf_counter()
61  err = False
```

```
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        err = True
        break

    x_new = x - (val / grad)
    X = np.linspace(x_new-1, x+1, 100)
    gradient_lines.append([X, gradient_line_gen(X, x,
       coefs)])
    absis_singgung.append(x)
    ordinat_singgung.append(val)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f}
       Gradien: {grad:.6f}')
    if (absol(x_new - x) < epsilon) and (absol(val) <
       epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k
           } iterasi.')
        err = True
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
if err:
    quit()
# gambar kurva
print('Menggambar kurva...')
batas_kiri = cari_batas_kiri(gradient_lines)
batas_kanan = cari_batas_kanan(gradient_lines)
X = np.linspace(batas_kiri, batas_kanan, 1000)
y_pol = function(X, coefs)
plt.figure(figsize=(10,9))
plt.plot(X, y_pol, color='#FE2C55', linewidth=2)
```

```
plt.axhline(0, color='white', linewidth=2)
plt.scatter(absis_singgung, ordinat_singgung, s=20, c="#25F4EE")
for lines in gradient_lines:
    plt.plot(lines[0], lines[1], color='#7DF9FF', linestyle='--', linewidth=1)
for i in range(len(absis_singgung)):
    plt.vlines(x=absis_singgung[i], ymin=0, ymax=ordinat_singgung[i], color='#8B008B', linewidth=2, linestyles='--')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Simulasi Algoritma Newton-Raphson')
plt.grid(True)
plt.rcParams['axes.linewidth'] = 10
plt.savefig('no1/ss_newton_raphson.png', dpi=300, edgecolor='none')
plt.show()
```

3.2 Algoritma Pinjam Buku

Kode 8: Kode Python simulasi algoritma Pinjam Buku

```
import time
import turtle
import random
"""Pinjam Buku (Simulasi)"""
# definisi fungsi
def cari_kandidat(koneksi, umur, sudah_pinjam, N):
    # pindah peminjam
    i = 0
    kandidat = -1
    umur_min = 999999
    while i < N:
        # cek koneksi dan status pinjam
        if (koneksi[i] == 1) and (sudah_pinjam[i] == 0):
            # Cek umur termuda
            if umur[i] < umur_min:
                umur_min = umur[i]
                kandidat = i
        i += 1
    return kandidat
# program utama
```

```
# input
N = int(input("Masukkan jumlah orang: "))
matrix = []
umur = [0 for _ in range(N)]
sudah_pinjam = [0 for _ in range(N)]
riwayat_pinjam = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Masukkan isi matrix ketetanggaan untuk koordinat ({x},{y}): '))
        matrix[y][x] = val
    print('================================')

for i in range(N):
    val_umur = int(input(f'Masukkan umur untuk index {i}: '))
    umur[i] = val_umur

index_pinjam = int(input('Masukkan index tempat awal buku berada: '))
# inisialisasi jendela labirin
layar = turtle.Screen()
layar.bgcolor("#1a1a1a")
t = turtle.Turtle()
t.hideturtle()
t.penup()
coordinates = []
warna_merah = True
for i in umur:
    x_sim = random.randint(-150, 150)
    y_sim = random.randint(-150, 150)
    coordinates.append([x_sim, y_sim])
    t.goto(x_sim, y_sim)
    if warna_merah:
        t.dot(10, '#FE2C55')
    else:
        t.dot(10, '#25F4EE')
    warna_merah = not warna_merah
# proses
```

```
awal = time.perf_counter()
sudah_pinjam[index_pinjam] = 1
riwayat_pinjam += [index_pinjam]
top = 0
t.goto(coordinates[index_pinjam][0], coordinates[
   index_pinjam][1])
t.pencolor('white')
t.shape('square')
t.pensize(2)
t.showturtle()
t.pendown()
print('Urutan index peminjaman buku: ', end='')
print(index_pinjam, end='')
while True:
    koneksi = matrix[index_pinjam]
    kandidat = cari_kandidat(koneksi, umur, sudah_pinjam
       , N)
    if kandidat != -1:
        index_pinjam = kandidat
        sudah_pinjam[index_pinjam] = 1
        riwayat_pinjam += [index_pinjam]
        top += 1
        t.goto(coordinates[index_pinjam][0], coordinates
           [index_pinjam][1])
        print(" ->", index_pinjam, end="")
    else:
        # Pengecekan apakah semua orang sudah pernah
           meminjam
        jumlah = 0
        for items in sudah_pinjam:
            jumlah += items
        # selesai hanya jika semua orang sudah pernah
           meminjam dan buku kembali ke peminjam asal
        if top == 0 and jumlah == N:
            break
        # Jika belum selesai tapi mentok, backtrack ke
           orang sebelumnya
        top -= 1
        index_pinjam = riwayat_pinjam[top]
        t.goto(coordinates[index_pinjam][0], coordinates
           [index_pinjam][1])
        print(" ->", index_pinjam, end="")

```

```
akhir = time.perf_counter()
eks = akhir - awal
print(f'\n\nWaktu eksekusi: {eks} detik.')
print('Menyimpan screenshot...')
canvas = turtle.getscreen().getcanvas()
canvas.postscript(file="no2/ss_pinjam_buku.eps")
layar.mainloop()
```

3.3 Algoritma Labirin

Kode 9: Kode Python simulasi algoritma Labirin

```
import time
import turtle
"""labirin (Simulasi)"""
# definisi fungsi
def choose_dir(pointer):
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
        direction = [1,1,1,1]
    return direction
def ruang_layar(x,y,N):
    xprime = (360//N)*(x-3)
    yprime = (360//N)*(-y+3)
    koordinat_layar = [xprime, yprime]
```

```
29      return koordinat_layar
30  # program utama
31  # input
32  N = int(input("Masukkan ukuran labirin: "))
33  matrix = []
34  visual_labirin = []
35  for i in range(N):
36      col = [0]*N
37      matrix += [col]
38      visual_labirin += [col]
39
40  for y in range(N):
41      for x in range(N):
42          val = int(input(f'Apakah koordinat ({x},{y})
                bisa dilewati (1) atau tidak (0): '))
43          matrix[y][x] = val
44      print('================================')
45
46  # inisialisasi jendela labirin
47  layar = turtle.Screen()
48  layar.bgcolor("#1a1a1a")
49  t = turtle.Turtle()
50  t.hideturtle()
51  t.penup()
52  for y in range(N):
53      for x in range(N):
54          t.goto(ruang_layar(x,y,N)[0], ruang_layar(x,y,N)
                [1])
55          if matrix[y][x] == 1:
56              t.dot(7, '#25F4EE')
57          else:
58              t.dot(7, '#FE2C55')
59  # proses
60  start_time = time.perf_counter()
61  # inisialisasi pointer dan arahnya
62  status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
63  # cari titik awal di sisi atas
64  for i in range(N):
65      if matrix[0][i] == 1:
66          status_sisi[0] = [i,0]
67          break
68  # cari titik awal di sisi bawah
69  for i in range(N):
```

```
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
t.goto(ruang_layar(pointer[0],pointer[1],N)[0],
   ruang_layar(pointer[0],pointer[1],N)[1])
t.pencolor('white')
t.shape('square')
t.pensize(5)
t.showturtle()
t.pendown()
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    t.goto(ruang_layar(pointer[0],pointer[1],N)[0],
       ruang_layar(pointer[0],pointer[1],N)[1])
    # cek arah yang tersedia
    direction = choose_dir(pointer)
```

```
    # cek apakah bisa dilewati
    i = 0
    while(i < 4):
        if direction[i] == 1:
            if (i == 0) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                    [1]-1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 1) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                    pointer[0]+1] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 2) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                    [1]+1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 3) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                    pointer[0]-1] == 1)
                avail_move[i] = bukan_dinding
        else:
            avail_move[i] = 0
        i += 1
    # Jika jalan buntu
    if avail_move == [0,0,0,0]:
        i = 0
        while(i < 4):
            if direction[i] == 1:
                if i == 0:
                    bukan_dinding = int(matrix[pointer
                        [1]-1][pointer[0]] == 1)
                    avail_move[i] = bukan_dinding
                elif i == 1:
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]+1] == 1)
                    avail_move[i] = bukan_dinding
                elif i == 2:
                    bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
                    avail_move[i] = bukan_dinding
                else:
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]-1] == 1)
```

```
                    avail_move[i] = bukan_dinding
            else:
                avail_move[i] = 0
            i += 1

    # Pilih jalur
    if clockwise:
        j = 0
        while (j < 4):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j += 1
    else:
        j = 3
        while (j >= 0):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
```

```
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j -= 1

    if (change_clockwise % (4*N)) == 0:
        clockwise = not clockwise

print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil dilewati')
t.goto(ruang_layar(pointer[0],pointer[1],N)[0], ruang_layar(pointer[0],pointer[1],N)[1])
end_time = time.perf_counter()
execution_time = end_time - start_time
print('Labirin Diselesaikan!')
print(f"Waktu eksekusi: {execution_time:.6f} detik")
print('Menyimpan screenshot...')
canvas = turtle.getscreen().getcanvas()
canvas.postscript(file="no3/ss_labirin.eps")
layar.mainloop()
```

4 Berbagai Link

4.1 Link Repositori GitHub

https://github.com/jovanphysics/praktikum-berkom-sem1